

Hackathon Problem Statement 1:

Explained Simply

Title: *Optimizing Urban Heat Mitigation and Cooling Strategies via Artificial Intelligence and Machine Learning (AI/ML)*

1. What is the Core Problem? (In Plain English)

Have you ever noticed that downtown city centers, asphalt parking lots, and dense concrete neighborhoods feel significantly hotter than a quiet, tree-lined suburb or park nearby?

This phenomenon is known as the **Urban Heat Island (UHI) effect**.

- Concrete, asphalt, and building roofs absorb heat during the day and radiate it back, making cities dangerously hot.
- Cities lack enough trees, vegetation, and water bodies to naturally cool the air through shade and evaporation.

The Challenge:

City planners cannot just plant trees randomly everywhere—budgets and space are limited.

They need to know:

1. **Where** are the worst heat hotspots right now?
2. **Why** is that specific neighborhood so hot? (Is it too many concrete buildings? Lack of vegetation? Low wind flow?)
3. **What can we do about it?** If we paint 20 roofs white, add a park, or plant 500 trees in

Sector 4, **how many degrees (°C) will the temperature drop?**

Your goal is to build an **AI-powered decision-support tool / web app** that detects these hotspots, explains what causes them, and simulates cooling strategies.

2. Breaking Down the 4 Core Deliverables

Stage	What You Need To Do	Practical Hackathon Meaning
1. Identify Hotspots	Generate heat stress maps using satellite & weather data.	Take satellite data (thermal infrared) and display a color-coded map (red = boiling, green = cool) of a target city.
2. Analyze Drivers	Quantify what causes the heat.	Use feature importance or statistical analysis to tell the

		user: <i>"In Zone B, 65% of excess heat is caused by low vegetation (NDVI) and 25% by high building density."</i>
3. Model Dynamics (AI/ML)	Train a model connecting city features to surface temperature (LST).	Train an ML model (e.g., XGBoost, Random Forest, or a CNN/GNN) where inputs are: land cover, building density, wind, vegetation; and target output is: Land Surface Temperature (LST) .
4. Cooling Simulator	"What-If" scenario builder & optimization.	Allow a city planner to select an area, change building roofs to cool roofs or add trees (modifying the input feature), and have the model predict: <i>"Target temperature reduces by 2.3°C."</i>

3. Demystifying the Datasets & Terms

The problem statement lists technical datasets. Here is what they actually mean and how to get them:

A. Remote Sensing Data (Satellites)

- **Land Surface Temperature (LST) from Landsat 8 / ECOSTRESS:**
 - Satellites equipped with thermal infrared sensors that measure the temperature of the ground/roofs (not air temperature).
 - *How to get:* Easily accessible via **Google Earth Engine (GEE)** or NASA Earthdata.
- **Land Use / Land Cover (LULC) from Sentinel-2 / Landsat:**
 - High-resolution optical satellite imagery showing whether a pixel is concrete, grass, water, forest, or bare soil.
 - You can extract common indices like **NDVI** (Normalized Difference Vegetation Index - measures greenness) and **NDBI** (Built-up Index).

B. Meteorological Data (Weather)

- **ERA5 / CPCB:**
 - **ERA5:** European climate reanalysis dataset containing hourly air temperature, humidity, and wind speed worldwide.
 - **CPCB (Central Pollution Control Board):** Ground-level weather and air quality monitoring stations across Indian cities.

C. Urban Form & Infrastructure

- **OpenStreetMap (OSM):**
 - Open-source street, road, and building footprint data. You can download building heights, road widths, and building densities using Python libraries like osmnx.
- **Global Human Settlement Layer (GHSL):**
 - Built-up area spatial grids showing population and building concentrations.

D. Optional Tools Mentioned

- **SOLWEIG & InVEST:** Existing open-source physics simulation tools for solar radiation and urban cooling. *Note: For a hackathon, you do NOT have to master both. You can use an ML surrogate model trained to mimic these principles.*

4. Suggested System Architecture & Tech Stack

A straightforward, high-impact architecture for your team:

[Satellite / Weather Data]

| (Google Earth Engine / OpenStreetMap API)



[Data Processing & Feature Extraction]

├— NDVI (Greenery Index)
├— NDBI (Concrete/Built-up Index)
├— Building Footprints & Density
└— Ambient Air Temp & Wind Speed



[AI / ML Core Engine]

├— Regression Model (XGBoost / LightGBM / Random Forest)
| Predicted LST = $f(\text{NDVI}, \text{NDBI}, \text{Building_Density}, \text{Wind}, \text{Albedo})$
└— Explainable AI (SHAP values to show "Drivers of Heat")



[Interactive Web Dashboard (Streamlit / React + Leaflet / Mapbox)]

├— 1. Interactive Heatmap of the City
├— 2. Driver Breakdown (Charts / SHAP plots)
└— 3. "What-If" Cooling Simulator (Sliders for adding trees / cool roofs)

5. Step-by-Step Execution Plan for Your Team

Phase 1: Pick One City (Don't try to solve the entire globe!)

- Pick a well-documented city with severe heat challenges (e.g., Delhi, Ahmedabad, Hyderabad, or Bengaluru).
- Defining a bounded geographic box ($10 \text{ km} \times 10 \text{ km}$) makes data retrieval and processing 10x faster.

Phase 2: Data Extraction (Use Google Earth Engine)

- Use the Python geemap or ee (Google Earth Engine) library to extract:
 1. Mean summer Land Surface Temperature (LST) for your chosen city.
 2. NDVI (Vegetation) and NDBI (Built-up index).
 3. Extract building density using osmnx.

Phase 3: Train the Baseline AI Model

- Create a tabular dataset where each row is a grid cell or neighborhood polygon with:
 - **Features:** Vegetation index, Built-up index, Albedo (reflectivity), Elevation, Wind speed.
 - **Target Label:** Observed Land Surface Temperature ($^{\circ}\text{C}$).
- Train an **XGBoost** or **Random Forest Regressor**.
- Run **SHAP (SHapley Additive exPlanations)** on the model. This satisfies the objective: *"Analyze & Quantify Drivers of Urban Heating"*.

Phase 4: Implement the Scenario Simulator (The "Winning" Feature)

- Give the user a simulated scenario:
 - *Action:* User increases vegetation (NDVI) by 30% in Ward 4.
 - *Inference:* Feed the modified feature vector into your trained model.
 - *Result:* Display: *"Predicted temperature reduction: 1.8°C "*.

Phase 5: Build a Clean UI

- Build the frontend with **Streamlit** (fastest for Python teams) or **React + Mapbox GL / Folium**.
- Show an interactive map with toggles:
 - Layer 1: Satellite True Color
 - Layer 2: Thermal Heatmap (Hotspots)
 - Layer 3: Simulation Output (Before vs. After intervention)

6. What Judges Will Look For (Evaluation Criteria)

1. **Working End-to-End Demo:** A live dashboard where judges can click a hotspot and see

what happens when trees or reflective roofs are added.

2. **Physics / Domain Awareness:** Acknowledging that planting trees requires water, or that cool roofs (high albedo) work best on flat concrete roofs.
3. **Actionable Recommendations:** Moving beyond just showing a pretty heatmap to answering: *"Where should the municipal corporation invest their next \$100,000 for maximum cooling?"*